Platform Engineering Take-Home Challenge

Role: Platform Engineer (L2 / L3) — Platform Engineering & Developer Experience **Time budget:** ~1 week, async. We expect roughly **8–12 hours of focused work**; please don't burn a whole week on this. If you spend significantly more or less, just tell us — it's useful context, not a penalty. **Submission:** Link to a public (or invited-private) GitHub repo + a 10-slide presentation (PDF or link).

Welcome

Hey! Thanks for taking the time to work on this with us.

A quick note before you start: **this challenge has no single "correct" answer.** We've seen great submissions solve this with very different architectures, and we care far more about *how* you reason through the problem than whether you picked the same services we would have. Be opinionated, make tradeoffs explicit, and don't be afraid to tell us what you chose *not* to do and why.

The Scenario

The engineering teams that support our **Web and Mobile apps** need help building a **user events tracking system**. This system captures every meaningful action a user takes in either app — `sign_in`, `purchase_completed`, `balance_checked`, `profile_updated`, and so on. Events are plain JSON objects.

They've come to the Platform Engineering team to design and operate the infrastructure that:

1. **Receives** these events from the Web and Mobile clients.
2. **Persists** them durably so the Data Engineering and Data Science teams can later analyze them to understand user behavior.

Your job is to design and implement that infrastructure.

Functional Requirements

- A mechanism (endpoint, SDK pattern, ingestion flow — your call) for the client apps to send events.
- Durable persistence of every accepted event.

- The persisted events must be **queryable or exportable** in a way that's reasonable for downstream Data teams to consume. You don't have to build the analytics layer — just make sure you haven't painted them into a corner.

Non-Functional Requirements

Your solution should account for:

- **High availability** — the ingestion path should not be a single point of failure.
- **Scalability** — it should handle traffic spikes without manual intervention.
- **Security** — think authentication, authorization, data in transit, data at rest, and blast radius of a compromised client.
- **Cost effectiveness** — reasonable for the scale described, with a clear sense of where money goes.

Implementation Requirements

- **Infrastructure as Code** — Terraform strongly preferred. If you use something else (CDK, Pulumi, CloudFormation), justify it in the presentation.
- **Language** — Python, JavaScript, or TypeScript for any application code.
- **Cloud provider** — AWS.
- **Compute model** — your choice: **fully serverless** (Lambda, Fargate-on-demand, etc.), **container-based** (ECS, EKS), or a **hybrid**. Pick what fits the problem and be ready to defend *why* you picked it over the alternatives. We genuinely don't have a preferred answer here — both directions have strong submissions in our history.
- **CI/CD** — include a minimal pipeline (GitHub Actions is fine) that at a minimum:
  - Lints and format-checks your IaC (`terraform fmt -check`, `tflint`, or equivalent).
  - Runs `terraform validate` and/or a `plan` on pull requests.
  - Deploys the solution on merge to main (or documents clearly how it *would*, if you chose the non-deploy path).
  - Bonus: a security scan step (`tfsec`, `checkov`, `trivy` — any credible choice).
- **Repository** — everything lives in a single GitHub repo with a clear README.

Scale Assumptions (for sizing and cost)

- **1,000,000 events per day**
- **~1.5 KB per event** (≈1.5 GB/day, ~45 GB/month raw)
- Traffic is not perfectly flat — assume reasonable daily peaks.

Deliver or Deploy? Your Call.

You can choose either:

- **(A) Actually deploy it** to a personal AWS account and share endpoint / console screenshots. Higher confidence signal for us, more cost and effort for you.
- **(B) Ship IaC + code that *could* be deployed**, with a clear README explaining how. Faster to produce, but you should compensate by making the code review crystal clear (structure, tests, `terraform plan` output, etc.).

Neither is penalized. Tell us which path you chose and why in your README. If you go with (B), we may run `terraform validate` / `terraform plan` ourselves against a sandbox.

Deliverables

1. GitHub repository

At minimum we expect:

- IaC for the full solution.
- Application code (producers, consumers, Lambda handlers, container definitions — whatever your design calls for).
- A **CI/CD workflow** (e.g., `.github/workflows/`) that lints and validates the IaC, and deploys the solution on merge (or documents the deploy step clearly if you chose not to deploy).
- A `README.md` that covers: architecture overview, how to deploy, how to send a test event, how to query persisted data, and any assumptions you made.
- An `ARCHITECTURE.md` or equivalent with a diagram (Flowchart Maker & Online Diagram Software, Excalidraw, Mermaid, napkin photo — we don't care about the tool).

2. A 10-slide presentation

Bring this to the follow-up interview. Suggested structure (flex as you see fit):

1. Problem statement in your own words
2. Requirements and assumptions
3. High-level architecture diagram
4. Ingestion path — design and tradeoffs
5. Persistence path — design and tradeoffs
6. How the non-functional requirements are met (HA, scalability, security)
7. **Cost estimate** at the stated scale (required — show your math)

8. What you'd improve with more time / at 10× scale
9. Operational story — observability, on-call, failure modes
10. Q&A / open discussion

Stretch Requirements — L3 candidates only

If you're interviewing for L3, please also address the following. L2 candidates are welcome to attempt these but won't be evaluated on them.

- **10× scale design:** describe (doesn't need to be implemented) how the architecture changes at **10M events/day**. What's the first thing to break? What would you change?
- **Schema evolution:** events will change over time. How do you handle additive fields, breaking changes, and versioning without blocking producers or breaking consumers?
- **Multi-region or DR posture:** at least discuss it — what's your RTO/RPO story? Is active/active worth it here? Why or why not?
- **Governance & guardrails:** how would you enforce standards across this system as other teams start sending events? Tagging, SCPs, budget alerts, CI checks — pick your poison and explain.
- **One ADR** (Architecture Decision Record) in the repo documenting your most consequential choice — the format is up to you, but we should be able to read it and understand the decision, the alternatives, and the consequences.

Ground Rules & FAQ

How polished does the code need to be? Production-intent, not production-perfect. We'd rather see clean structure, sensible tests on the critical path, and honest `TODO`s than a gold-plated submission that took you 40 hours.

Can I use AI assistants (Copilot, Claude Code, Cursor, etc.)? Yes — we use them ourselves. Use them the way you would on the job: lean on them, but own every line you commit. Be ready to discuss and defend any part of the codebase in the interview.

Can I ask clarifying questions? Absolutely. Email the recruiter — we'd rather answer a question than have you guess. Also fine to make an assumption and document it in the README; we'll read those carefully.

What if I don't finish? Ship what you have with a clear "what I didn't get to and why" section. We'd much rather see a smaller, honest, well-reasoned submission than a sprawling incomplete one. Scope-cutting is a platform engineering skill.

Do I have to use AWS-managed services? No. Self-hosted is fair game if you can justify it. Just know that "why not the managed option?" will come up in the interview.

What's the follow-up interview like? ~60 minutes. You walk us through your slides (~20–25 min), we dig into the design and code (~25–30 min), and you ask us questions (~10 min). It's a conversation, not a defense.

What We're Looking For (at a high level)

- Clear thinking about tradeoffs — every design choice closes some doors and opens others. Tell us which.
- Appropriate scope — over-engineering is as much a red flag as under-engineering.
- Operational awareness — how does this thing behave at 3am on a Sunday?
- Communication — your README and slides are part of the evaluation.

Have fun with it. Good luck — we're rooting for you.